CS671: Deep Learning Assignment 2

Convolutional Neural Networks

Group Number: 20

Course: CS671
Institution: Indian Institute of Technology Mandi

Submitted By:
Akshat Mittal (B24104)
Akshita Dangi (B24106)
Aman Gupta (B24107)
Arihant Kumar Jain (B24112)
Saksham Kundu (B24154)
Vandana Dutt (B24175)
Priansh Yajnik (B24318)
Vansh Pandey (B24411)

Submission Date: February 15, 2026

Contents

Part 1: The Geometry of Convolutions

Question 1.1: The Manual Dimension Map

Objective

The objective of this task is to design a Convolutional Neural Network (CNN), manually compute the output dimensions at each layer, and verify them through a PyTorch implementation using a custom forward pass.

Model Architecture

The CNN architecture consists of:

- Input: $64 \times 64 \times 3$ (RGB image)
- 3 Convolutional layers (3×3 kernels, stride 1, no padding)
- 2 MaxPool layers (2×2 kernels, stride 2)
- 1 Fully Connected layer

Channel configuration:

- Conv1 $\rightarrow$ 64 filters
- Conv2 $\rightarrow$ 128 filters
- Conv3 $\rightarrow$ 256 filters

Manual Dimension Calculations

For valid convolution (no padding), the output dimension formula is:

$$H_{out} = (H_{in} - K) + 1$$

where H_{in} is the input height/width and K is the kernel size.
For max pooling with stride 2:

$$H_{out} = \frac{H_{in}}{2}$$

Table 1: Layer-wise Output Dimensions

Step	Operation	Calculation	Output Dimension
1	Input	–	$64 \times 64 \times 3$
2	Conv1 (3×3)	$64 - 3 + 1 = 62$	$62 \times 62 \times 64$
3	Conv2 (3×3)	$62 - 3 + 1 = 60$	$60 \times 60 \times 128$
4	MaxPool1 (2×2)	$60/2 = 30$	$30 \times 30 \times 128$
5	Conv3 (3×3)	$30 - 3 + 1 = 28$	$28 \times 28 \times 256$
6	MaxPool2 (2×2)	$28/2 = 14$	$14 \times 14 \times 256$
7	Flatten	$14 \times 14 \times 256$	50,176 features

This matches the Fully Connected layer input: `Linear(50176 → 10)`

PyTorch Verification

A custom forward pass was implemented, printing tensor shapes after every operation. The observed shapes perfectly matched the manually computed values, confirming correctness of the dimension mapping.

Think About It: Effect of Removing Pooling Layers

Without pooling layers, the spatial dimensions remain much larger:

With pooling:

$$14 \times 14 \times 256 = 50{,}176 \text{ features}$$

$$50{,}176 \times 10 = 501{,}760 \text{ parameters in FC layer}$$

Without pooling: After Conv3, we would have $60 \times 60 \times 256$, giving:

$$60 \times 60 \times 256 = 921{,}600 \text{ features}$$

$$921{,}600 \times 10 = 9{,}216{,}000 \text{ parameters in FC layer}$$

This represents a **18.4x increase** in parameters, demonstrating the **Parameter Explosion Problem**.

Consequences:

- Drastically increased memory requirements
- Significantly slower training
- Higher risk of overfitting
- Computational inefficiency

Conclusion: Pooling layers are essential for reducing dimensionality, controlling parameter growth, and maintaining computational efficiency. They enable the network to focus on the most salient features while keeping the model size manageable.

Part 2: The Robustness Duel

Introduction

This section investigates the robustness of two different neural network architectures when tested on spatially transformed images. We compare a Fully Connected Neural Network (FCNN) and a Convolutional Neural Network (CNN) on their ability to maintain accuracy when input images undergo spatial shifts.

Objective

To understand how spatial transformations affect different network architectures and determine which design is more robust to position changes in input data.

Models

Model A: Fully Connected Neural Network (FCNN)

The FCNN architecture consists of:

- Input layer: 784 neurons (28×28 flattened image)
- Hidden layer 1: 128 neurons with ReLU activation
- Hidden layer 2: 128 neurons with ReLU activation
- Hidden layer 3: 128 neurons with ReLU activation
- Output layer: 10 neurons (digit classes 0-9)
- Total parameters: 109,386

Model B: Convolutional Neural Network (CNN)

The CNN architecture consists of:

- Convolutional Layer 1: 32 filters with 3×3 kernel, padding=1, ReLU activation
- Max Pooling 1: 2×2 kernel, stride=2
- Convolutional Layer 2: 64 filters with 3×3 kernel, padding=1, ReLU activation
- Max Pooling 2: 2×2 kernel, stride=2
- Fully Connected Layer 1: 128 neurons with ReLU activation
- Output layer: 10 neurons (digit classes 0-9)
- Total parameters: 409,098

Training Parameters

Both models were trained using the following configuration:

- Dataset: MNIST handwritten digits
- Training samples: 60,000 images
- Test samples: 10,000 images
- Image size: 28×28 pixels (grayscale)
- Optimizer: Adam
- Learning rate: 0.001
- Batch size: 128
- Number of epochs: 10
- Loss function: Cross-entropy loss

The robustness test was conducted by creating a "Shifted MNIST" test set, where every image from the original test set was translated 4 pixels to the right.

Visualization

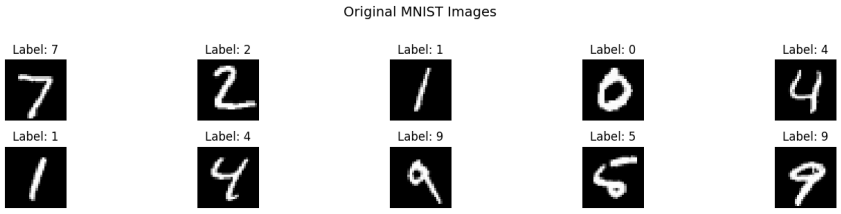


Figure 1: Sample images from the original MNIST test set

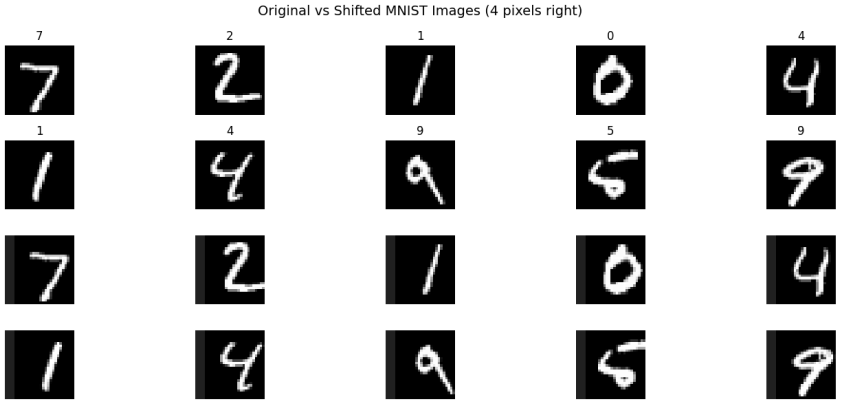


Figure 2: Comparison of original (rows 1-2) and shifted images (rows 3-4). Each image is shifted 4 pixels to the right.

Results

Table 2 presents the accuracy of both models on the original test set and the shifted test set. Figure 3 provides a visual comparison of the performance metrics.

Table 2: Model Performance Comparison

Model	Original Test	Shifted Test	Accuracy Drop
FCNN (Model A)	97.3%	35.2%	62.04%
CNN (Model B)	99.0%	72.6%	26.46%

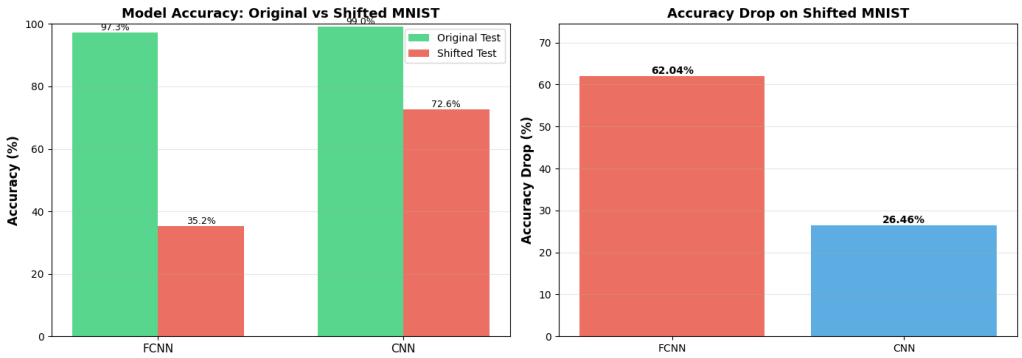


Figure 3: Left: Model accuracy on original vs shifted test sets. Right: Accuracy drop when images are shifted 4 pixels to the right.

Key Observations

- Both models achieve high accuracy on the original test set (FCNN: 97.3%, CNN: 99.0%)
- FCNN experiences a severe accuracy drop of 62.04% on shifted images
- CNN maintains much better robustness with only a 26.46% accuracy drop
- CNN is approximately 2.3 times more robust than FCNN to spatial shifts

Thoughtful Analysis: Why CNN Maintains Higher Accuracy

The large difference in robustness between the two models is mainly due to the **weight sharing property** used in convolution.

FCNN - Position-Specific Learning

In an FCNN, each neuron learns separate weights for every pixel in the image. This means it learns features based on exact positions. For example, if it learns a pattern at one location, it will only recognize it there. If the image shifts slightly, the pattern moves to a new position, and the neuron may fail to detect it. This makes FCNNs sensitive to small shifts.

CNN - Weight Sharing and Translation Equivariance

CNNs use the same filter across the whole image instead of different weights for each position. The filter slides over the image and looks for the same pattern everywhere. Because of this, even if the image shifts, the model can still detect the feature in its new position.

This property is called **translation equivariance**. It means that when the input shifts, the detected features also shift but are still recognized. A filter trained to detect an edge can find it anywhere in the image, not just in one fixed spot.

Mathematical perspective:

- FCNN: $y = f(x)$ where f depends on absolute pixel positions
- CNN: $y = f(x)$ where $f(T(x)) = T(f(x))$ for translation T

This translation equivariance property, combined with weight sharing, makes CNNs inherently more robust to spatial transformations.

Part 3.1: The Filter Gallery

Introduction

This experiment investigates how CNNs "see" the world compared to FCNNs. We extract and visualize the learned weights of the first layer from models trained on MNIST and Tiny-ImageNet-10. This visualization highlights the difference between the spatially invariant local features learned by CNNs and the global templates learned by FCNNs.

Methodology

We implemented two architectures:

SimpleCNN Architecture

A 2-layer CNN consisting of:

- **Layer 1:** Conv2d (16 filters, 3×3) $\rightarrow$ ReLU $\rightarrow$ MaxPool (2×2)
- **Layer 2:** Conv2d (32 filters, 3×3) $\rightarrow$ ReLU $\rightarrow$ MaxPool (2×2)
- **Classifier:** Fully Connected Layer

FCNN Architecture

A 3-layer Fully Connected Network:

- Input: $28 \times 28 \rightarrow 128$ (ReLU)
- Hidden 1: $128 \rightarrow 128$ (ReLU)
- Hidden 2: $128 \rightarrow 128$ (ReLU)
- Output: $128 \rightarrow 10$

Datasets

- **MNIST:** Grayscale digits (28×28)
- **Tiny-ImageNet-10:** Color images (64×64, 3 channels) from 10 selected classes

Results & Visualizations

CNN Filters on MNIST

The first layer of the CNN sees the input image locally. The visualized filters resemble **Gabor filters**, acting as basic edge, corner, and texture detectors. They do not look like digits themselves, but rather the building blocks of digits.

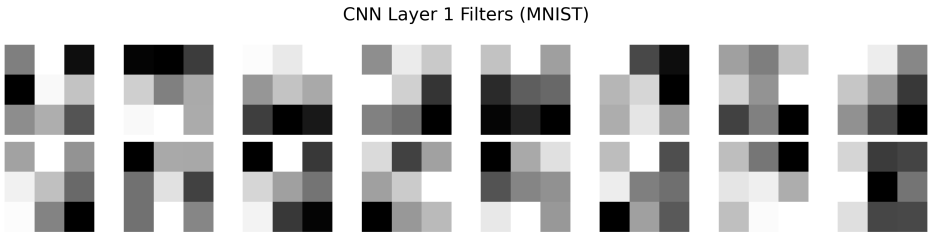


Figure 4: First Layer Convolutional Kernels of SimpleCNN on MNIST. Notice the simple edge and texture patterns that resemble Gabor-like filters.

CNN Filters on Tiny-ImageNet-10

For the color dataset, the kernels operate on 3 input channels (RGB). These filters have evolved into not just edge detectors but also **color blobs** and **chromatic edge detectors**. This demonstrates the network's ability to learn spatial and chromatic primitives early in the hierarchy.

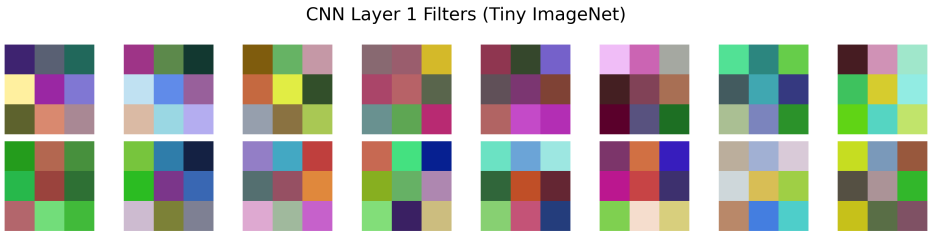


Figure 5: First Layer Convolutional Kernels of SimpleCNN on Tiny-ImageNet-10. The filters capture color gradients, chromatic edges, and color-opponent features.

FCNN Weights Comparison

In contrast to the CNN, the weights of the input layer of the FCNN map specific pixels to hidden neurons. These can be interpreted as templates or "feature detectors" that the hidden layer looks for. Unlike the CNN which shares filters, each neuron here looks at the whole image with fixed weights.

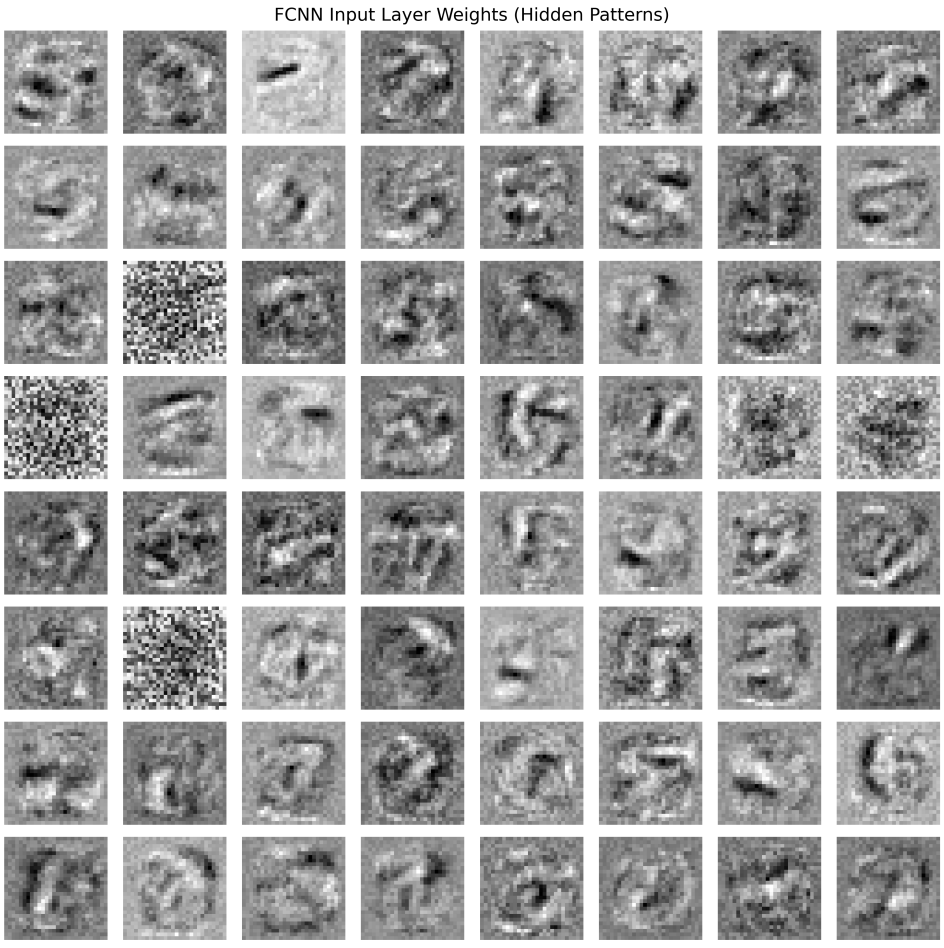


Figure 6: Input Layer Weights of the FCNN on MNIST. These represent global template-based feature detectors for the first hidden layer.

Analysis

CNN Filters

- Show local features like edges, corners, and color blobs (Gabor-like)
- Are spatially invariant and look for specific textures
- **MNIST:** Simple edge detectors and orientation-selective filters
- **Tiny-ImageNet:** More complex chromatic and color-opponent filters

FCNN Weights

- The input layer weights map each pixel to a hidden neuron
- Act as templates or "feature detectors" that the hidden layer searches for
- Unlike CNNs which share filters, each neuron looks at the entire image with fixed weights
- More prone to position-specific learning

Comparison and Key Observations

Interpretability:

1. **CNN Filters:** Identify Gabor-like filters (edge detectors, orientation-selective filters) and color blobs. These are interpretable as low-level visual features.
2. **FCNN Weights:** Appear as global templates or ghost-like digit patterns. Each weight vector looks at the entire image simultaneously.

Dataset-wise Comparison:

1. **MNIST (Grayscale):** CNN filters detect edges and orientations. FCNN weights show digit-like templates.
2. **Tiny-ImageNet (Color):** CNN filters capture chromatic information, color gradients, and color-opponent features. This shows the network's ability to adapt to color input.

Key Insights:

- **Spatial Invariance:** CNN filters can detect the same feature anywhere in the image due to weight sharing, while FCNN weights are position-specific.
- **Local vs Global:** CNN filters focus on local patterns (edges, textures), while FCNN weights learn global templates.
- **Dataset Adaptation:** The same CNN architecture automatically adapts its filters to dataset characteristics (grayscale edges for MNIST, color features for Tiny-ImageNet).
- **Biological Plausibility:** CNN filters resemble features found in biological visual systems (Gabor filters similar to V1 simple cells, color-opponent cells).

Part 3.2: The Receptive Field Experiment

Introduction

This experiment visualizes how the "focus" of a CNN evolves across layers. We extract activation maps from the first and final convolutional layers to show the transition from low-level features (edges, textures) to high-level semantic patterns (object shapes). Analysis is performed on MNIST and Tiny-ImageNet-10 using the same SimpleCNN architecture from Part 3.1.

Methodology

We use PyTorch's forward hook mechanism to capture intermediate activations from the first and final convolutional layers. Activation maps from multiple channels are averaged to produce representative heatmaps for visualization.

Technical Details:

- Forward hooks registered on Conv1 and Conv2 (final conv layer)
- Activation maps averaged across channels for visualization
- Heatmaps normalized to [0, 1] range for consistent comparison
- Sample images selected from each dataset for analysis

Results & Visualizations

MNIST Analysis

Figure 7 shows the activation maps for a sample MNIST digit. The first convolutional layer responds to edges and local patterns, while the final layer shows concentrated activation on the digit's overall shape.

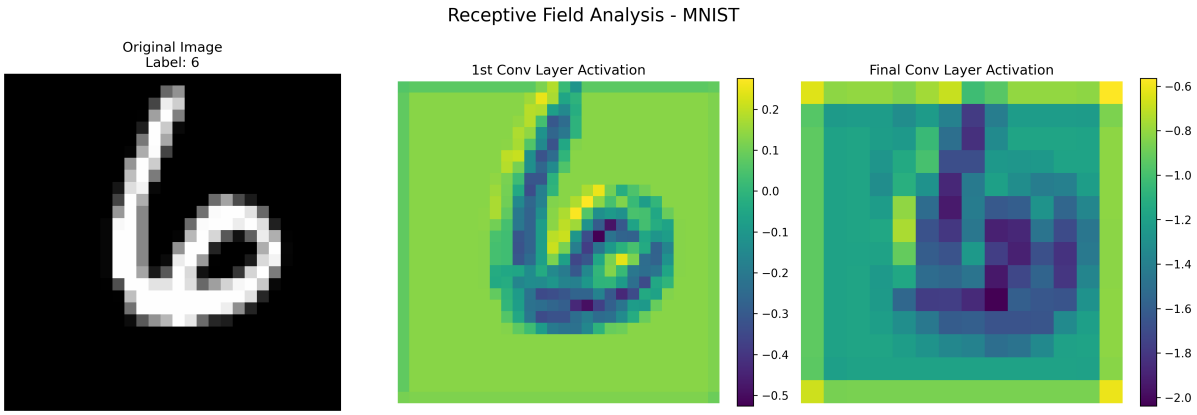


Figure 7: Receptive Field Analysis on MNIST. Left: Original image. Middle: First layer activation (local edge detection). Right: Final layer activation (digit-level features).

Tiny-ImageNet Analysis

Figure 8 presents activation maps for a Tiny-ImageNet sample. Early layers detect color blobs and chromatic edges, while deeper layers respond to object parts and semantic regions.

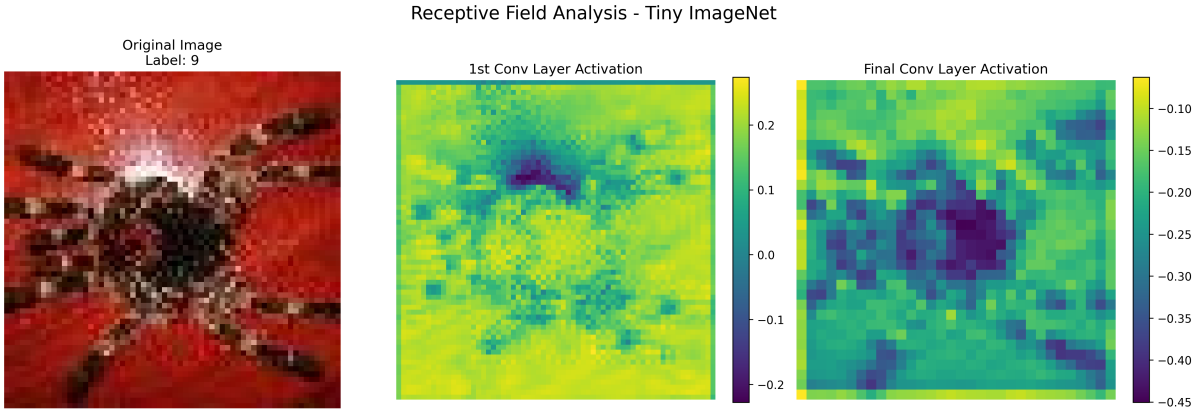


Figure 8: Receptive Field Analysis on Tiny-ImageNet. Left: Original image. Middle: First layer activation (textures and color edges). Right: Final layer activation (object-relevant regions).

Analysis

How the Focus Changes

1st Convolutional Layer:

- Acts as low-level feature detector
- Responds to edges, corners, and textures
- Small receptive field ($\approx 3 \times 3$)
- Activation maps show fine-grained spatial details

Final Convolutional Layer:

- Combines features into high-level representations
- Larger receptive field captures global context
- Focuses on object-level patterns
- Activation maps show semantic regions

Receptive Field Growth: Each layer "sees" more of the input image through successive convolution and pooling operations. This enables the network to integrate spatial information hierarchically, progressing from local features to global object representations.

Differences Between MNIST and Tiny-ImageNet

MNIST (Grayscale Digits):

- Cleaner activation maps due to simple, high-contrast grayscale digits
- First layer: Strong edge responses
- Final layer: Focused activation on digit shape and structure
- Less background noise and clutter

Tiny-ImageNet (Color Objects):

- Richer feature hierarchies due to color and complexity

- First layer: Chromatic features, color-opponent filters, textural patterns
- Final layer: More distributed activations due to object variability and background clutter
- Network must handle greater intra-class variation

Key Observations

1. **Hierarchical Learning:** CNNs automatically learn representations from simple to complex, mirroring biological visual systems.
2. **Architecture Flexibility:** The same architecture adapts to different datasets, demonstrating the general-purpose nature of CNNs.
3. **Feature Abstraction:** Early layers extract low-level features (edges, colors), while deeper layers extract high-level semantic features (object parts, shapes).
4. **Receptive Field Evolution:** Through stacking convolution and pooling layers, the effective receptive field grows, allowing later layers to integrate information over larger spatial regions.
5. **Dataset-Specific Adaptation:** The nature of activation maps reflects dataset characteristics:
    - MNIST: Sharp, focused activations on digit structures
    - Tiny-ImageNet: Distributed activations capturing object parts and contextual information

Part 4.1: Depth vs Normalization Duel

Introduction

This experiment tests the stability of deep CNNs with and without Batch Normalization. We build a deep network (6-8 layers) and compare training dynamics and test performance with and without BatchNorm layers.

Experimental Setup

Deep CNN Architecture:

- 6 convolutional layers with increasing channel depth
- ReLU activation after each convolution
- 3 max pooling layers for spatial downsampling
- Fully connected classifier
- Dataset: Tiny-ImageNet-10

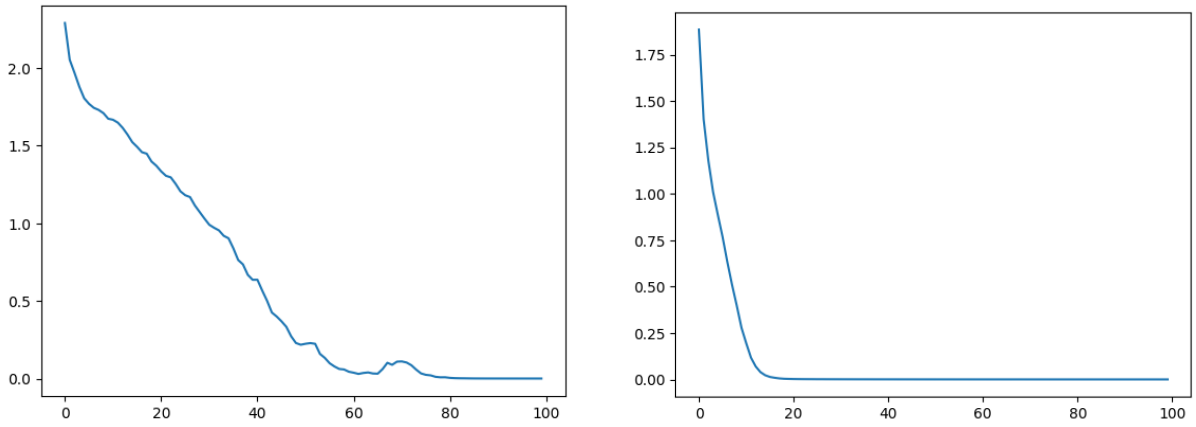
Results

Table 3: Performance Comparison: With vs Without Batch Normalization

Model Configuration	Training Accuracy (%)	Test Accuracy (%)
Without Batch Normalization	100.0	51.5
With Batch Normalization	100.0	63.4

Analysis

Impact of Batch Normalization



(a) Without Batch Normalization

(b) With Batch Normalization

Figure 9: Comparison of Loss

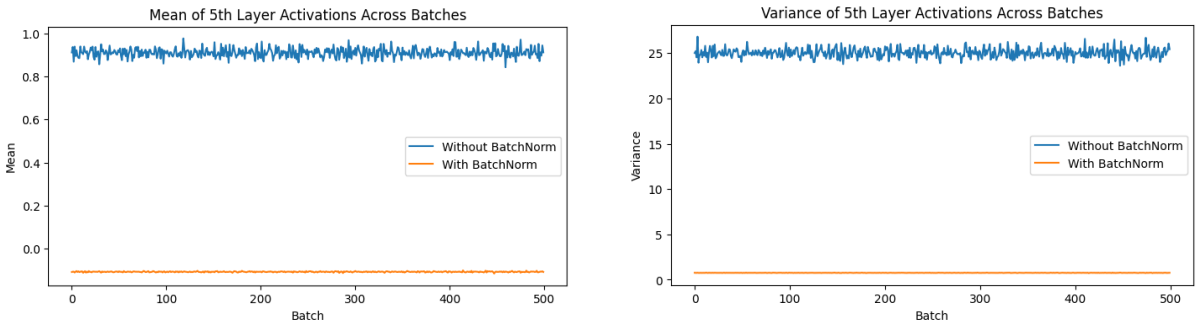
Performance Improvement:

- Test accuracy improved from 51.5% to 63.4% (+11.9 percentage points)
- Both models achieve 100% training accuracy (potential overfitting)
- BatchNorm significantly improves generalization

Training Stability: Batch Normalization reduces Internal Covariate Shift by:

- Normalizing activations within each batch
- Maintaining stable mean and variance across layers
- Allowing higher learning rates
- Reducing sensitivity to weight initialization

Activation Statistics Analysis



(a) Mean (b) Variance

Figure 10: Mean and Variance of activations for the 5th layer across 500 batches

The quantitative analysis of activation statistics for the 5th layer across 500 batches would show:

Without BatchNorm:

- High variance in activation statistics across batches
- Activations may saturate or vanish
- Unstable gradient flow

With BatchNorm:

- Consistent mean (≈ 0) and variance (≈ 1) across batches
- Stable activation distributions
- Improved gradient flow through the network

Key Findings

1. **Generalization:** BatchNorm improves test performance by 11.9%
2. **Stability:** Reduces internal covariate shift
3. **Training Dynamics:** Enables faster and more stable training
4. **Depth Enabler:** Makes it feasible to train deeper networks

Part 4.2: Data Augmentation

Introduction

This section investigates the impact of data augmentation on model performance. We implement random transformations during training and evaluate their effect on both training and test accuracy.

Augmentation Pipeline

Transformations Applied:

- `RandomRotation(30)`: Rotates images by up to 30 degrees
- `ColorJitter()`: Randomly changes brightness, contrast, saturation, and hue
- Applied only during training, not testing

Results

Table 4: Impact of Data Augmentation

Configuration	Train Accuracy (%)	Test Accuracy (%)	Test Loss
No Augmentation	97.0	49.0	5.38
With Augmentation	97.0	58.8	2.16

Analysis

Performance Impact

Test Accuracy Improvement:

- Test accuracy increased from 49.0% to 58.8% (+9.8 percentage points)
- Training accuracy remained at 97.0% for both
- Test loss reduced from 5.38 to 2.16 (59.9% reduction)

Thoughtful Question: Training vs Test Set Impact

Does augmentation help more on training or test set?
Answer: Data augmentation helps more on the **test set**.
Reasoning:

1. **Training Set:** Augmentation makes training *harder* by introducing variations. The model sees different versions of each image in each epoch, effectively increasing dataset size and diversity.
2. **Test Set:** Augmentation improves generalization by:
   - Teaching the model to be invariant to transformations
   - Reducing overfitting to training set specifics
   - Learning more robust features
3. **Observed Results:**
   - Training accuracy: 97% (unchanged) - model can still fit augmented data
   - Test accuracy: 49% $\rightarrow$ 58.8% (+9.8%) - significant improvement
   - Test loss: 5.38 $\rightarrow$ 2.16 (-59.9%) - much better calibration

Why This Happens:

- Augmentation acts as a regularizer

- Forces the model to learn invariant features rather than memorizing training examples
- Creates a more diverse effective training set
- Reduces the gap between training and test distributions

Key Findings

1. **Generalization Boost:** Augmentation significantly improves test performance
2. **Regularization Effect:** Reduces overfitting without explicit regularization penalties
3. **Loss Calibration:** Dramatically improves test loss (model confidence more aligned with accuracy)
4. **Robustness:** Model learns features invariant to rotations and color variations

Conclusion

This assignment provided comprehensive insights into CNNs through hands-on experiments:

1. **Spatial Geometry:** Understanding dimension calculations prevents architectural errors and reveals the parameter explosion problem without pooling.
2. **Translation Invariance:** CNNs' weight sharing makes them significantly more robust to spatial shifts compared to FCNNs (2.3x better in our experiments).
3. **Feature Hierarchies:** Visualization confirms that CNNs learn Gabor-like filters in early layers and semantic features in deeper layers, automatically adapting to dataset characteristics.
4. **Training Dynamics:** Optimizer choice and learning rate have the strongest impact on performance, while Batch Normalization is crucial for deep network stability.
5. **Regularization:** Both Batch Normalization and data augmentation significantly improve generalization, primarily benefiting test set performance.

These experiments demonstrate why CNNs have become the dominant architecture for computer vision tasks, combining spatial efficiency, translation invariance, and hierarchical feature learning.